

Main code 분석

Class Lechateliersimulator:

르사틀리에 원리를 표현하는 메인 UI 클래스입니다.

def __init__(self, root):

클래스가 인스턴스화 될 때 자동 실행되는 생성자입니다.

인스턴스 뜻: 프로그래밍에서 클래스를 기반으로 생성된 객체

self.Engine= LechatelierEngine()

화학 평형 이동의 계산을 담당하는 LechatelierEngine의 인스턴스를 생성하여 결합합니다.

인스턴스: 클래스라는 설계도를 기반으로 실제로 만들어진 구체적인 객체

self.notebook=tk.Notebook(self.root)

self.notebook.pack(fill="both", expand=True, padx=8, pady=8)

창 크기가 변할 때 탭 영역도 가로/세로 전체로 늘어나도록 (fill="both", expand=True) 배치.

tab_sim= tk.Frame(self.notebook, bg="#f5f6fa")

tab_quiz= tk.Frame(self.notebook, bg="white")

화면을 위한 흰색 배경을 설치합니다.

self.notebook.add(tab_sim, text=" 시뮬레이터")

self.notebook.add(tab_quiz, text=" 퀴즈 모드")

생성한 프레임들을 화면에 추가하고, 탭 상단에 표시될 텍스트와 이미지를 지정합니다.

self.setup_ui(tab_sim)

시뮬레이터 내부에 입력창, 그래프 등을 배치하기 위하여 setup_ui 함수를 불러옵니다.

self.quiz_manager=

LeChatelierQuizManager(tab_quiz)

클래스 객체를 가동, (tab_quiz)제어

self.load_default_reaction()

초기 데이터 주입 함수를 호출하여 시뮬레이션 작동

input_frame=tk.LabelFrame(parent, text=" 1. 반응식 및 엔탈피 조건 설정", bg="#f5f6fa",

font=("맑은 고딕", 11, "bold"))

시각적으로 보이기 위하여 테두리와 제목이 적힌 그룹 상자 위젯을 생성함

input_frame.pack(fill="x", padx=15, pady=(15,5))

상자를 가로 방향으로 배치한뒤 좌우 15 픽셀, 상단 15 픽셀, 하단 5픽셀

tk.Label(input_frame, ...)

사용자가 잘못 입력하는것을 방지하기위한 안내 글씨를 띄움

grid(row=0, column=0, ...)

안내 자막을 상자 내부 0번째 행, 열에 배치후 5칸을 차지하여 왼쪽에 배치시킨다.

tk.Label(input_frame, text="반응식: "...)
입력창 왼쪽에 반응식: 이름표 글씨 라벨을 만듦

grid(row=1, column=0, padx(10,2)...)
이름표를 1번 행, 0번 열 배치

self.entry_reaction= tk.Entry(input_frame, width=3*, font=("맑은 고딕", 10))
사용자가 화학 평형 반응식을 적을수 있도록 35글자 텍스트 입력창을 생성한다

self.entry_reaction.grid(row=1, column=1, padx=5, pady=5)
입력창을 이름 표 바로 오른쪽 1번 행, 1번 열 자리 배치

self.entry_reaction.grid(row=1, column=1, padx=5, pady=5)
화학식 입력창을 1번 행, 1번 열 좌표에 정렬함

self.var_deltah=tk.StringVar(value="exothermic")
발열 또는 흡열 선택 버튼의 체크 상태를 실시간으로 기억하고 제어하는 Tkinter 전용 특수 문자열 변수를 선언하고 기본값을 발열로 둠

dh_frame=tk.Frame(input_frame, bg=...)
버튼 두개를 잘보이게 나열하기위해 1행 2열 자리에 깔아둘 상자를 생성함

dh_frame.grid(row=1, column=2...)
버튼용 소형 프레임 산자를 1번 행, 2번째 열 자리에 배치

rb_exo= tk.Radiobutton(dh_frame...)
엔탈피 부호가 음수임을 알려주기위한 빨간색 글씨를 사용하여 발열 선택 버튼 생성
rb_endo= tk.Radiobutton(dh_frame...)
엔탈피 부호가 양수임을 알려주기위한 파란색 글씨를 사용하여 연산 함수 발동
rb_exo.pack(side="left", padx=5)
발열 버튼을 프레임 내부에서 왼쪽부터 배열

rb_endo.pack(side="left", padx=5)
흡열 버튼을 프레임 내부에서 왼쪽부터 배열

tk.Button(input_frame, text="반응식 적용", ...)
텍스트를 확정하는 파란색 버튼을 생성

grid(row=1, cloumn=3, padx=5, pady=5)
반응식 적용 버튼을 1번 행, 3번 열에 배치

self.con_input_frame...
각 기체의 초기 농도를 적을 수 있는 위젯들이 생성됨

Self.con_input_frame.grid(row=2, ...)
2번 행에 길게 왼쪽 정렬로 정렬합니다

control_frame=tk.LabelFrame(parent, text"2. 외부 조건 변화 조절 (르샤틀리에 조작)...

실험 조작 변인들을 묶어줄 두 번째 테두리 상자를 만든다

```
control_frame.pack(fill="x", padx=15, pady=5)
```

이 조작 상자도 가로 길게 배치한다

```
tk.Label(control_frame, text="온도 조절:", ...
```

온도 조절: 글씨를 0행 0열에 오른쪽 (sticky="e")하여 배치합니다

```
self.scale_temp= ttk.Scale(control_frame, from=-5, to=5...)

드래그마다 함수를 가동하여 계산한뒤 온도 슬라이더 바를 만든다
```

```
self.scale_temp.grid(row=0, cloumn=1, padx=5, pady=5)
```

온도 슬라이더 바를 0번 행, 1번열 배치

```
self.lbl_temp.grid(row=0, column=2, padx=5, pady=5, sticky="w")
```

온도를 0번 행, 2번 열에 배치

```
tk.Label(control_frame, text"용기 부피 압축(압력):", ...
```

용기 부피 압축, 글씨를 1행 0열에 오른쪽 배치

```
self.scale_pres=ttk.Scale(control_frame, from=-5, to=5...)

드래그마다 함수로 계산한뒤 압력 슬라이더 바 생성
```

```
self.scale_pres.grid(row=1, column=1, padx=5, pady=5)
```

압력 슬라이더 바를 1번 행, 1번 열 자리에 배치

```
Self.lbl_pres=tk.Label(control_frame, text="기본상태 ...
```

현재 압축 상태에 따른 기압 수치 출력

```
self.lbl_pres.grid(row=1, cloumn=2, padx=5, pady=5, sticky="w")
```

기압을 1번 행, 2번 열 배치

```
tk.Label(control_frame, text="특정 물질 가변 조작:", ...
```

특정 물질 가변 조작, 글씨를 2행 0열 배치

```
self.combo_chemical= ttk. Combobox...
```

조작할 기체를 선택하는 메뉴 생성

```
self.combo_chemical.grid(row=2, column=1, sticky="w", padx=5, pady=5)
```

메뉴판을 2번 행, 1번열 배치

```
self.combo_chemical.bind...
```

기체를 바꾸어 고를때 연산 함수가 켜지도록 생성

```
self.scale_chem= ttk.Scale(control_frame, from=-2, to=2,...)
```

선택한 기체 농도를 조절가능한 슬라이더바 생성

```
self.scale_chem.grid(row=2, column=1, sticky"e", padx=(120, 5), pady=5)
```

농도 슬라이더 바를 겹치지 않도록 2번 행, 1번 열에 배치

```
self.lbl_chem= tk.Label(control_frame, text="변화 없음...")
내가 조작한 특정 기체의 농도 변화 수치를, 띄어주는 창 생성
```

```
self.lbl_chem.grid(row=2, column=2, padx=5, pady=5, sticky="w")
농도 변화 창을 2번째, 2번 열에 배치
```

```
tk.Button(control_frame, text="조건 초기화",...)
모든 실험 조건을 원점으로 초기화하는 버튼 생성. 함수 생성
```

```
grid(row=0, column=3, rowspan=3, padx=20, pady=5, ipady=10)
초기화 버튼은 3열에 배치하고, 3칸의 행을 세로로 병합하여 간격을 늘린다
```

```
display_frame= tk.Frame(parent, bg="#f5f6fa")
베이스 프레임 생성
```

```
display_frame.pack(fill="both", expand=True, padx=15, pady=5)
이 베이스가 화면에 꽉 차도록 확장 배치
```

```
vessel_box= tk.LabelFrame(display_frame, text="3. 실제 반응 용기 내부 (농도 시각화)...")
반응 용기를 그리기 위한 흰색 테두리 상자를 만든다
```

```
vessel_box.pack(side="left", fill="both", expand=True, padx=(0,50), pady=5)
이 상자를 왼쪽에 배치하고 화면 크기에 밀착하여 늘어나도록 함
```

```
self.canvas= tk.Canvas(vessel_box, ...)
넓은 색깔 화면 생성, 테두리 선 두께는 강제 제거
```

```
self.canvas.pack(fill="both", expand=True, padx=10, pady=10)
화면 내부에 빈틈없이 꽉 차도록 확장 배치
```

```
report_box= tk.LabelFrame(display_frame, text="4. 평행 이동 분석 결과", ...)
평행 이동 현상을 서술하여 가로 폭 280 픽셀 크기 상자 제작
```

```
report_box.pack(side="right", fill="both", padx=(5,0), pady=5)
이 상자를 가로 방향 오른쪽에 배치
```

```
self.lbl_status= tk.Label(report_box, text="평형 상태 확인 중...",
bg="white"...
현재 "정반응 진행", "역반응 진행" 인지를 한눈에 보여주는 커다란 상태창 생성
```

```
self.lbl_status.pack(fill="x", padx=10, pady=15)
상태창을 보고서 상자 내부 맨 위 배치
```

```
self.txt_report= tk.Text( report_box, bg=...
연산 결과 화학 열역학 이론 보고서가 적힌 텍스트 창입니다.
```

```
self.txt_report.pack(fill="both", expand=True, padx=10, pady=(0,10))
```

보고서 창 내부 남은 하단 공간 전체를 꽉 채워 배치

```
self.entry_reaction.insert(0, "N2+3H2⇌2NH3")
```

프로그램이 실행되면 하버 공정 반응식 글자를 채워 넣습니다

```
self.parse_reaction()
```

글자가 채워진후 화학식을 분해를 위하여 함수를 가동합니다

```
self.engine.parse_reaction_logic(self.entry_reaction.get(), self.combo_chemical, self.con_input_frame)
```

입력창에 적힌 글자와 드롭 다운 위젯, 하부 컨테이너 프레임을 매개변수로 묶고 그대로 전송함. 문자열에서 기호 추적후 분해한 뒤 반응물과 생성물 계수 정보 추출

```
self.reset_sliders()
```

새로운 화학식이 왔으니 슬라이더를 원점으로 돌린다

```
self.engine.is Updating=True
```

슬라이더 수치를 코드로 리셋 하면 슬라이더가 자동으로 작동후 연산 함수를 사용한다. 발생하는 렉을 막기 위하여 is Updating=True라는 '연산 잠금 플래그 차단막' 을 켜두는 technique 입니다

```
self.scale_temp.set(0)
```

```
self.scale_pres.set(0)
```

```
self.scale_chem.set(0)
```

온도, 압력, 농도 슬라이더의 위치를 초기인 0으로 강제로 리셋함

```
self.engine.is Updating=False
```

정렬 이 끝났으므로 연산 잠금 차단막을 해제 합니다

```
self.trigger_simulation()
```

아무 조작을 하지 않은 기본 평형 상태인 0, 0, 0 인 상태에서 최초로 그리도록 함수를 부른다

```
self.scale_temp.get()
```

```
self.scale_pres.get()
```

```
self.scale_chem.get()
```

온도 변화, 압축률, 농도 실수형 수치를 마우스 포인터 그 자체의 위치에서 수집

```
self.combo_chemical.get(), self.var_deltah.get()
```

기체 종류 이름과 엔탈피 버튼을 불러옴

```
self.lbl_temp, ...
```

연산 완료, 결과 텍스트를 적은뒤 그래픽을 직접 렌더링하는 출력용 위젯들의 제어권임

```
self.engine.calculate_equilibrium_shift(...)
```

수치와 변수들을 패키징 과정을 거친 후 르샤틀리에 시뮬 계산을 최종적으로 수행

참고를 위한 원래 main code

```

class LeChatelierSimulator:
    def __init__(self, root):
        self.root = root
        self.root.title("화학 평형 이동 시뮬레이터 (르샤틀리에의 원리)")
        self.root.geometry("1020x750")
        self.root.configure(bg="#f5f6fa")

        # 팀원 B의 엔진 결합
        self.engine = LeChatelierEngine()

        # Notebook 탭 레이아웃 분할
        self.notebook = ttk.Notebook(self.root)
        self.notebook.pack(fill="both", expand=True, padx=8, pady=8)

        tab_sim = tk.Frame(self.notebook, bg="#f5f6fa")
        tab_quiz = tk.Frame(self.notebook, bg="white")

        self.notebook.add(tab_sim, text="🔬 시뮬레이터 ")
        self.notebook.add(tab_quiz, text="📄 퀴즈 모드 ")

        # 각 구역 레이아웃 렌더링 호출
        self.setup_ui(tab_sim)

        # [🔥 중요] 팀원 C가 만든 퀴즈 통합 객체를 퀴즈 탭 프레임에 전달하여 초기화
        self.quiz_manager = LeChatelierQuizManager(tab_quiz)

        self.load_default_reaction()

    def setup_ui(self, parent):
        # --- 1구역: 반응식 설정 패널 ---
        input_frame = tk.LabelFrame(parent, text="1. 반응식 및 엔탈피 (ΔH) 조건 설정 ",
                                     bg="#f5f6fa", font=("맑은 고딕", 11,
                                     "bold"))
        input_frame.pack(fill="x", padx=15, pady=(15, 5))

        tk.Label(input_frame, text="입력 예시: N2 + 3H2 <=> 2NH3 (우측에서 열역학 엔탈피 부호를 직접 선택하세요)",
                  bg="#f5f6fa", fg="#7f8c8d", font=("맑은 고딕", 9)).grid(row=0, column=0, columnspan=5, sticky="w", padx=10, pady=2)

        tk.Label(input_frame, text="반응식:", bg="#f5f6fa", font=("맑은 고딕", 10)).grid(row=1, column=0, padx=(10, 2), pady=5)

        self.entry_reaction = tk.Entry(input_frame, width=35, font=("맑은 고딕", 10))
        self.entry_reaction.grid(row=1, column=1, padx=5, pady=5)

        self.var_deltah = tk.StringVar(value="exothermic")
        dh_frame = tk.Frame(input_frame, bg="#f5f6fa")
        dh_frame.grid(row=1, column=2, padx=10, pady=5)

```

```

        rb_exo = tk.Radiobutton(dh_frame, text="ΔH < 0 (발열)",
                                variable=self.var_deltah,
                                value="exothermic", bg="#f5f6fa",
                                font=("맑은 고딕", 9, "bold"), fg="#e74c3c",
                                command=self.trigger_simulation)
        rb_endo = tk.Radiobutton(dh_frame, text="ΔH > 0 (흡열)",
                                variable=self.var_deltah,
                                value="endothermic", bg="#f5f6fa",
                                font=("맑은 고딕", 9, "bold"), fg="#2980b9",
                                command=self.trigger_simulation)
        rb_exo.pack(side="left", padx=5)
        rb_endo.pack(side="left", padx=5)

        tk.Button(input_frame, text="반응식 적용",
                  command=self.parse_reaction,
                  bg="#3498db", fg="white", font=("맑은 고딕", 9,
                  "bold")).grid(row=1, column=3, padx=5, pady=5)

        self.con_input_frame = tk.Frame(input_frame, bg="#f5f6fa")
        self.con_input_frame.grid(row=2, column=0, columnspan=5,
        sticky="w", padx=10, pady=5)

        # --- 2구역: 변인 컨트롤 프레임 ---
        control_frame = tk.LabelFrame(parent, text=" 2. 외부 조건 변화 조절
        (르샤틀리에 조작) ",
                                      bg="#f5f6fa", font=("맑은 고딕",
        11, "bold"))
        control_frame.pack(fill="x", padx=15, pady=5)

        tk.Label(control_frame, text="온도 조절:", bg="#f5f6fa",
        font=("맑은 고딕", 10)).grid(row=0, column=0, padx=10, pady=5,
        sticky="e")
        self.scale_temp = ttk.Scale(control_frame, from_=-5, to=5,
        orient="horizontal", length=300, command=self.trigger_simulation)
        self.scale_temp.grid(row=0, column=1, padx=5, pady=5)
        self.lbl_temp = tk.Label(control_frame, text="기본 상태 (25.00
        °C)", bg="#f5f6fa", fg="#2c3e50", font=("맑은 고딕", 9, "bold"),
        width=25, anchor="w")
        self.lbl_temp.grid(row=0, column=2, padx=5, pady=5, sticky="w")

        tk.Label(control_frame, text="용기 부피 압축(압력):", bg="#f5f6fa",
        font=("맑은 고딕", 10)).grid(row=1, column=0, padx=10, pady=5,
        sticky="e")
        self.scale_pres = ttk.Scale(control_frame, from_=-5, to=5,
        orient="horizontal", length=300, command=self.trigger_simulation)
        self.scale_pres.grid(row=1, column=1, padx=5, pady=5)
        self.lbl_pres = tk.Label(control_frame, text="기본 상태 (1.00
        atm)", bg="#f5f6fa", fg="#2c3e50", font=("맑은 고딕", 9, "bold"),
        width=25, anchor="w")
        self.lbl_pres.grid(row=1, column=2, padx=5, pady=5, sticky="w")

```

```

        tk.Label(control_frame, text="특정 물질 가변 조작:", bg="#f5f6fa",
font=("맑은 고딕", 10)).grid(row=2, column=0, padx=10, pady=5,
sticky="e")
        self.combo_chemical = ttk.Combobox(control_frame,
state="readonly", width=10)
        self.combo_chemical.grid(row=2, column=1, sticky="w", padx=5,
pady=5)
        self.combo_chemical.bind("<<ComboboxSelected>>", lambda e:
self.trigger_simulation())

        self.scale_chem = ttk.Scale(control_frame, from_=-2, to=2,
orient="horizontal", length=180, command=self.trigger_simulation)
        self.scale_chem.grid(row=2, column=1, sticky="e", padx=(120,
5), pady=5)

        self.lbl_chem = tk.Label(control_frame, text="변화 없음 (0.00
M)", bg="#f5f6fa", fg="#2c3e50", font=("맑은 고딕", 9, "bold"), width=25,
anchor="w")
        self.lbl_chem.grid(row=2, column=2, padx=5, pady=5, sticky="w")

        tk.Button(control_frame, text="조건 초기화",
command=self.reset_sliders,
                bg="#95a5a6", fg="white", font=("맑은 고딕", 9,
"bold")).grid(row=0, column=3, rowspan=3, padx=20, pady=5, ipady=10)

# --- 3/4구역: 디스플레이 패널 ---
        display_frame = tk.Frame(parent, bg="#f5f6fa")
        display_frame.pack(fill="both", expand=True, padx=15, pady=5)

        vessel_box = tk.LabelFrame(display_frame, text=" 3. 실제 반응 용기
내부 (농도 시각화) ", bg="white", font=("맑은 고딕", 11, "bold"))
        vessel_box.pack(side="left", fill="both", expand=True, padx=(0,
5), pady=5)

        self.canvas = tk.Canvas(vessel_box, bg="#1e272e",
highlightthickness=0)
        self.canvas.pack(fill="both", expand=True, padx=10, pady=10)

        report_box = tk.LabelFrame(display_frame, text=" 4. 평형 이동 분석
결과 ", bg="white", font=("맑은 고딕", 11, "bold"), width=280)
        report_box.pack(side="right", fill="both", padx=(5, 0), pady=5)

        self.lbl_status = tk.Label(report_box, text="평형 상태 확인
중...", bg="white", fg="#2980b9", font=("맑은 고딕", 12, "bold"),
justify="center")
        self.lbl_status.pack(fill="x", padx=10, pady=15)

        self.txt_report = tk.Text(report_box, bg="#f8f9fa", bd=0,
font=("맑은 고딕", 10), spacing1=3)

```

```

        self.txt_report.pack(fill="both", expand=True, padx=10,
pady=(0, 10))

    def load_default_reaction(self):
        self.entry_reaction.insert(0, "N2 + 3H2 <=> 2NH3")
        self.parse_reaction()

    def parse_reaction(self):
        self.engine.parse_reaction_logic(self.entry_reaction.get(),
self.combo_chemical, self.con_input_frame)
        self.reset_sliders()

    def reset_sliders(self):
        self.engine.is Updating = True
        self.scale_temp.set(0)
        self.scale_pres.set(0)
        self.scale_chem.set(0)
        self.engine.is Updating = False
        self.trigger_simulation()

    def trigger_simulation(self, *args):
        self.engine.calculate_equilibrium_shift(
            self.scale_temp.get(), self.scale_pres.get(),
self.scale_chem.get(),
            self.combo_chemical.get(), self.var_deltah.get(),
            self.lbl_temp, self.lbl_pres, self.lbl_chem,
self.lbl_status, self.txt_report, self.canvas
        )

# — 프로그램 실행 엔트리포인트 —————
if __name__ == "__main__":
    root = tk.Tk()
    app = LeChatelierSimulator(root)
    root.update()
    app.trigger_simulation()
    root.mainloop()

```